

Урок 18. Корутины. Каналы

14 августа 2020

В этом уроке разбираемся как работают каналы. Что под капотом, закрытие и отмена канала, буфер, produce, consume, broadcast.

Как и во всех предыдущих уроках здесь будет только теория с небольшими примерами. Реальные боевые примеры будем разбирать в отдельных уроках.

Канал используется, как средство передачи данных между корутинами. Т.е. он является ячейкой, куда одна корутина может поместить данные, а другая корутина - взять их оттуда.

Если вам знаком интерфейс `BlockingQueue` из Java, то канал работает по схожему принципу.

В самом простом случае это выглядит так:

```
1 val channel = Channel<Int>()
2
3 launch {
4     // ...
5     channel.send(5)
6     // ...
7 }
8
9 launch {
10    // ...
11    val i = channel.receive()
12    // ...
13 }
```

Мы создаем канал, указав тип данных, которые мы через него сможем передать - `Int`. Далее используем этот канал в двух разных корутинах. Одна из них методом `send` помещает в канал значение 5. А другая корутина получает это значение.

Сразу отмечу, что канал - потокобезопасен и нам не надо самим возиться с блокировками и синхронностью.

Давайте подробно обсудим поведение методов `send` и `receive`. Эти методы являются `suspend` функциями. Они могут отработать мгновенно, а могут и приостановить корутину на какое-то время. Это зависит от того, какой из них был вызван раньше.

Если первая корутина пытается отправить данные методом `send`, но вторая корутина еще не вызвала `receive`, то метод `send` приостановит первую корутину и будет ждать. Аналогично наоборот. Если вторая корутина уже вызвала `receive`, чтобы получить данные, но первая корутина еще не отправила их, то метод `receive` приостановит вторую корутину.

Рассмотрим два этих случая на примерах и поговорим о том, как это работает.

Под капотом

Рассмотрим случай, когда отправитель хочет отправить данные, но получатель еще не готов их принять.

```
1 val channel = Channel<Int>()
2
3 launch {
4     delay(300)
5     log("send 5")
6     channel.send(5)
7     log("send, done")
8 }
9
10 launch {
11     delay(1000)
12     log("receive")
13     val i = channel.receive()
14     log("receive $i, done")
15 }
```

Две корутины, стартующие в одно время и работающие параллельно. Первая корутина отправляет данные в канал через 300 мсек после запуска. Но вторая будет готова эти данные получать только через 1000 мсек. В итоге первая корутина будет приостановлена на 700 мсек.

Как это будет работать изнутри.

Первая корутина вызывает метод `send`, чтобы отправить данные. Метод `send` проверяет, есть ли получатель ожидающий данных. Т.е. кто-нибудь, кто вызвал метод `receive`. Такого пока нет. Значит, просто сразу передать данные уже ожидающему получателю не получится. Придется ждать вызова `receive`. Для этого надо уходить в полноценный `suspend` вызов.

Используется классическая схема создания `suspend` функции. Вызывается штука, похожая на `suspendCoroutine`, чтобы получить `continuation` текущей (первой) корутины. Данные (число 5) и `continuation` пакуются в контейнер `SendElement`, и этот контейнер сохраняется внутри канала. На этом вызов `send` завершен. Выполнение кода первой корутины приостановлено `suspend` функцией `send` пока кто-нибудь не выполнит `continuation.resume`.

Вторая корутина вызывает метод `receive`, чтобы получить данные. Метод `receive` проверяет, не отправлял ли кто данные. Для этого он смотрит на наличие в канале контейнера `SendElement`. Такой контейнер есть, т.к. метод `send` поместил его туда ранее. Из контейнера извлекаются данные и `continuation`. Данные используются в качестве результата вызова метода `receive`, т.е. вторая корутина получает эти данные. А у `continuation` вызывается `resume`, чтобы возобновить выполнение первой корутины.

Лог:
15:28:09.511 send 5
15:28:10.211 receive
15:28:10.211 receive 5, done
15:28:10.212 send, done

Видно, что метод `send` приостановил первую корутину на время ожидания получателя. Через 700 мсек получатель во второй корутине вызвал метод `receive` и мгновенно получил данные, т.к. отправитель эти данные уже поместил в канал. Забрав данные, получатель возобновил выполнение первой корутины.

Теперь рассмотрим обратный порядок вызовов, когда получатель уже готов, а отправитель еще ничего не отправлял.

```
1 val channel = Channel<Int>()
2
3 launch {
4     delay(1000)
5     log("send 5")
6     channel.send(5)
7     log("send, done")
8 }
9
10 launch {
11     delay(300)
12     log("receive")
13     val i = channel.receive()
14     log("receive $i, done")
15 }
```

Вторая корутина готова получать данные через 300 мсек после запуска, но первая готова отправить данные только через 1000 мсек. Вторая корутина будет приостановлена на 700 мсек.

Как это будет работать изнутри.

Вторая корутина вызывает метод `receive`, чтобы получить данные. Метод `receive` проверяет, не отправлял ли кто данные. Для этого он смотрит на наличие в канале контейнера `SendElement`. Такого контейнера нет, т.к. метод `send` еще не был вызван. Значит, сразу получить данные не получится. Придется ждать вызова `send`. Для этого надо уходить в полноценный `suspend` вызов.

Извлекается `continuation` текущей корутины и упаковывается в контейнер `ReceiveElement`. Этот контейнер сохраняется внутри канала. На этом вызов метода `receive` завершен. Выполнение кода второй корутины приостановлено, пока кто-нибудь не выполнит `continuation.resume`.

Первая корутина вызывает метод `send`, чтобы отправить данные. Метод `send` проверяет, нет ли уже ожидающего получателя. Для этого он смотрит на наличие в канале контейнера `ReceiveElement`. Такой контейнер есть, т.к. метод `receive` поместил его туда ранее. Из контейнера извлекается `continuation`, и вызывается его метод `resume` с передачей туда данных, которые были отправлены в метод `send` (т.е. число 5). Это и возобновляет вторую корутину и передает туда данные. В итоге метод `receive` возвращает данные, которые первая корутина передавала в метод `send`.

Лог:
15:29:33.466 receive
15:29:34.167 send 5
15:29:34.167 send, done
15:29:34.168 receive 5, done

Видно, что метод `receive` приостановил вторую корутину на время ожидания отправки. Через 700 мсек отправитель в первой корутине вызвал метод `send`, чтобы отправить данные, и, тем самым, возобновил выполнение второй корутины.

Потоки

В плане потоков методы `send` и `receive` ведут себя так же, как и обычные `suspend` функции. Они приостанавливают корутину, если им приходится ждать отправления или получения данных. Когда корутина возобновляется, она может продолжить свое выполнение в другом потоке. Но в любом случае это будет поток из диспетчера корутины.

Передача нескольких элементов

Через один канал можно отправлять данные сколько угодно раз.

Но тут надо не забывать о том, что каждый метод `send` ждет вызова `receive`. Если будет, например, 5 вызовов `send` и только 3 `receive`, то первая корутина остановится на 4-м вызове `send` и больше не продолжится.

```

1  launch {
2      // ...
3      channel.send(5)
4      // ...
5      channel.send(6)
6      // ...
7      channel.send(7)
8      // ...
9      channel.send(8)
10     // ...
11     channel.send(9)
12 }
13
14 launch {
15     // ...
16     var i = channel.receive()
17     // ...
18     i = channel.receive()
19     // ...
20     i = channel.receive()
21 }

```

Значение 8 не будет отправлено, т.к. получать его будет никому.

Аналогично наоборот. Если есть 5 вызовов receive, и только 3 вызова send, то четвертый receive не дожидается отправки данных и приостановит вторую корутину навсегда.

```

1  launch {
2      // ...
3      channel.send(5)
4      // ...
5      channel.send(6)
6      // ...
7      channel.send(7)
8  }
9
10 launch {
11     // ...
12     var i = channel.receive()
13     // ...
14     i = channel.receive()
15     // ...
16     i = channel.receive()
17     // ...
18     i = channel.receive()
19     // ...
20     i = channel.receive()
21 }

```

close

Очевидно, что корутинa получатель не может всегда заранее знать о количестве элементов, которые будут отправлены в канал. Хорошо бы дать отправителю возможность сообщать о том, что он закончил передавать данные. Получив такой сигнал получатель больше не вызывал бы метод receive, который приостановит корутину, но никогда ничего не вернет.

Для этого у отправителя есть возможность закрыть канал методом close.

```

1  launch {
2      // ...
3      channel.send(5)
4      // ...
5      channel.send(6)
6      // ...
7      channel.send(7)
8      // ...
9      channel.close()
10 }

```

Метод close - это явный сигнал о том, что данные больше передаваться не будут. В этом случае в канал помещается специальное служебное значение Closed. Если получатель попытается получить данные после закрытия канала (или уже ждет данные в момент закрытия), то метод receive получит этот Closed и выбросит исключение ClosedReceiveChannelException.

Чтобы нам самим не возиться с отловом и обработкой этого исключения, доступна такая форма получения данных (вместо метода receive):

```

1  launch {
2
3      for (element in channel) {
4          // ...
5      }
6
7  }

```

Этот цикл будет получать данные из канала, пока отправитель не закроет канал. После чего цикл завершится, и код пойдет дальше. И никакие ошибки ловить не придется.

Также, на стороне получателя мы можем использовать метод channel.isClosedForReceive, чтобы понять, что канал закрыт и данных в нем не осталось.

capacity

По умолчанию канал может хранить в себе только одно значение, пока получатель не заберет его. Это поведение можно поменять, используя параметр `capacity` при создании канала.

Фиксированное значение

Указав число, мы получим канал с буфером указанного размера.

Пример:

```
1 val channel = Channel<Int>(2)
2
3 launch {
4     repeat(7) {
5         delay(300)
6         log("send $it")
7         channel.send(it)
8     }
9     log("close")
10    channel.close()
11 }
12
13 launch {
14     for (element in channel) {
15         log("received $element")
16         delay(1000)
17     }
18 }
```

Создаем канал с буфером на 2 элемента.

Первая корутина отправляет 7 значений с паузой в 300 мсек. А вторая получает их с паузой 1000 мсек.

Лог:

```
14:32:32.238 send 0
14:32:32.239 received 0
14:32:32.553 send 1
14:32:32.854 send 2
14:32:33.156 send 3
14:32:33.247 received 1
14:32:33.549 send 4
14:32:34.248 received 2
14:32:34.550 send 5
14:32:35.250 received 3
14:32:35.564 send 6
14:32:36.253 received 4
14:32:36.253 close
14:32:37.264 received 5
14:32:38.266 received 6
```

Видно, что поначалу первая корутина отправляет значения не дожидаясь вызовов `receive` для каждого значения на стороне получателя. Значения сохраняются в буфере канала. Когда он заполняется, то каждому `send` снова приходится ждать, пока `receive` не возьмет одно значение и, тем самым, не освободит одно место в буфере. И в конце получатель забирает данные уже только из буфера.

Обратите внимание, что даже после закрытия канала (`close`), получатель сможет получить все данные из буфера.

Conflated

В этом режиме буфера нет. Канал способен хранить в себе только одно значение. Но теперь каждый `send` не ждет, пока получатель заберет значение, а просто оставляет его в канале. При этом `send` своими данными перезаписывает данные предыдущего `send`.

Пример:

```
1 val channel = Channel<Int>(Channel.Factory.CONFLATED)
2
3 launch {
4     repeat(7) {
5         delay(300)
6         log("send $it")
7         channel.send(it)
8     }
9     log("close")
10    channel.close()
11 }
12
13 launch {
14     for (element in channel) {
15         log("received $element")
16         delay(1000)
17     }
18 }
```

Лог:
14:37:35.657 send 0
14:37:35.658 received 0
14:37:35.958 send 1
14:37:36.261 send 2
14:37:36.562 send 3
14:37:36.664 received 3
14:37:36.865 send 4
14:37:37.165 send 5
14:37:37.468 send 6
14:37:37.468 close
14:37:37.673 received 6

Метод send не ждет вызовов receive, он просто помещает в канал свое значение, заменяя предыдущее. Метод receive получает значение, которое на момент вызова этого метода находится в канале.

Unlimited

Размер буфера в таком канале ограничен только количеством доступной памяти.

```
1 | val channel = Channel<Int>(Channel.Factory.UNLIMITED)
```

Buffered

Канал с размером буфера = 64 по умолчанию. Это значение можно поменять в JVM.

```
1 | val channel = Channel<Int>(Channel.Factory.BUFFERED)
```

cancel

Кроме закрытия канала, его можно отменить. Это то же самое, что и закрытие, только с очисткой буфера.

Рассмотрим тот же пример, что и ранее с фиксированным размером буфера. Но теперь будем не закрывать канал, а отменять его методом cancel.

```
1 | val channel = Channel<Int>(2)
2 |
3 | launch {
4 |     repeat(7) {
5 |         delay(300)
6 |         log("send $it")
7 |         channel.send(it)
8 |     }
9 |     log("cancel")
10 |    channel.cancel()
11 | }
12 |
13 | launch {
14 |     for (element in channel) {
15 |         log("received $element")
16 |         delay(1000)
17 |     }
18 | }
```

Лог:
14:34:54.596 send 0
14:34:54.597 received 0
14:34:54.898 send 1
14:34:55.199 send 2
14:34:55.501 send 3
14:34:55.599 received 1
14:34:55.901 send 4
14:34:56.601 received 2
14:34:56.902 send 5
14:34:57.603 received 3
14:34:57.904 send 6
14:34:58.605 received 4
14:34:58.605 cancel

В прошлом примере после закрытия канала получатель мог достать данные из буфера. А после отмены уже не может, значения 5 и 6 для получателя утеряны.

Еще одно отличие cancel от close в том, что при попытке вызвать receive для отмененного канала будет выброшено исключение CancellationException, а не ClosedReceiveChannelException

И цикл for:

```

1 launch {
2
3     for (element in channel) {
4         // ...
5     }
6
7 }

```

также выбросит `CancellationException`.

В уроке про отмену корутин мы обсуждали это исключение. Оно не приведет к крэшу, а просто отменит корутину, в которой оно произошло.

Получатель закрывает канал

Разумеется, отменить/закрыть канал может и корутина получателя. При этом в корутине отправителя метод `send` выбросит, соответственно, исключение `CancellationException` (что приведет к отмене корутины) / `ClosedSendChannelException` (что приведет к крэшу, если исключение никак не обрабатывается).

Отправитель может использовать метод `channel.isClosedForSend()`, чтобы определить, можно ли отправлять данные в канал.

Отмена корутины

Что будет, если отменить корутину-отправителя? Если метод `send` работает в `suspend` режиме (т.е. ему приходится ждать получателя), то отправка данных в канал будет отменена, а корутина завершится. Т.е. происходит стандартная реакция отменяемой `suspend` функции. Но канал при этом не будет отменен или закрыт. Т.е. получатель ничего не узнает и продолжит ждать.

Чтобы это исправить, как вариант, можно в корутине-отправителе явно попросить `job` завершать работу канала при завершении работы корутины.

```

1 launch {
2     coroutineContext[Job]?.invokeOnCompletion {
3         channel.close()
4     }
5     // ...
6 }

```

`invokeOnCompletion` вызовется как при нормальном завершении корутины, так и при отмене или ошибке.

Аналогично со стороны получателя. Если отменить корутину получателя, то получение будет отменено и корутина завершится. А отправитель (если он еще работает) будет продолжать пытаться отправить данные.

Несколько отправителей или получателей

Мы можем отправлять данные в канал из нескольких корутин. Или, наоборот, получать данные из канала в нескольких корутинах.

Простой пример:

```

1 val channel = Channel<Int>()
2
3 launch {
4     // ..
5     channel.send(1)
6     // ..
7     channel.send(2)
8     // ..
9 }
10
11 launch {
12     // ..
13     channel.send(11)
14     // ..
15     channel.send(12)
16     // ..
17 }
18
19 launch {
20     for (element in channel) {
21         // ...
22     }
23 }

```

Отправляем данные в канал из двух корутин. Тут, правда, возникает вопрос - кто будет закрывать канал? Это надо каким-то образом определить, что все отправители уже закончили работу и только потом вызвать `close`.

Если никто не вызовет `close`, то `for` цикл в корутине получателя не завершится. Конечно, когда отменят `scope` этой корутины, то цикл прервется и корутина завершится на этом месте. Но что если после этого цикла есть код, который должен быть выполнен? Значит `for` цикл должен быть завершен корректно. Значит канал надо закрывать на стороне отправителя.

Как вариант, можно все корутины отправители поместить в один отдельный родительский launch и в нем использовать invokeOnCompletion. Он будет выполнен, только когда выполнится родительский launch и все корутины внутри него. В нем вполне можно закрыть канал, чтобы уведомить получателя.

```
1  val channel = Channel<Int>()
2
3  launch {
4      coroutineContext[Job]?.invokeOnCompletion {
5          channel.close()
6      }
7
8      launch {
9          // channel.send(...)
10     }
11
12     launch {
13         // channel.send(...)
14     }
15 }
16
17 launch {
18     for (element in channel) {
19         // ...
20     }
21     // ...
22 }
```

produce

Вместо кода:

```
1  val channel = Channel<Int>()
2
3  launch {
4      // ...
5      channel.send(5)
6      // ...
7      channel.send(6)
8      // ...
9      channel.send(7)
10     // ...
11     channel.close()
12 }
```

Можно написать так

```
1  val channel = produce<Int> {
2      // ...
3      send(5)
4      // ...
5      send(6)
6      // ...
7      send(7)
8      // ...
9  }
```

Билдер produce создает и запускает отдельную корутину. Также он создает канал. Этот канал доступен в блоке кода напрямую. Т.е. мы пишем просто send, а не channel.send.

Билдер возвращает созданный канал, как ReceiveChannel. Этот интерфейс является частью интерфейса Channel:

```
1  interface Channel<E> : SendChannel<E>, ReceiveChannel<E>
```

ReceiveChannel - содержит методы, необходимые получателю, а *SendChannel* - отправителю.

Т.е. канал, который мы получаем от билдера produce, предназначен только для получения данных, не для отправки. Предполагается, что отправлять в него данные будет только блок кода билдера produce.

У этого симбиоза корутины и канала есть две важные особенности.

Во-первых, канал будет автоматически закрыт по выполнению блока кода. Нам не надо самим вызывать close.

Во-вторых, если отменить корутину, в который был вызван produce, то канал также отменится. Соответственно, если где-то там наверху отменен scope или одна из корутин выше по иерархии, то отмена доберется и до этого канала.

consume

Есть еще один вариант получения данных из канала:

```
1 | channel.consumeEach { element ->
2 |     // ...
3 | }
```

Под капотом тут просто try-catch обертка над for циклом, который мы рассматривали ранее. Если эта обертка поймает ошибку (в нашем коде обработки получаемых данных), то она отменит канал.

Это приведет к тому, что другие получатели этого канала больше ничего не получат. Этим consumeEach и отличается от обычного for цикла. Если ошибка произойдет в обычном for цикле, то этот цикл просто прервется сам, но не будет отменять канал, и остальные получатели продолжат получать данные.

BroadcastChannel

Все данные, отправляемые в обычный канал будут равномерно распределены между получателями. Т.е. если получатель один - он получит все данные. А если получателей несколько, то данные будут разделены между ними, и каждый получит только часть от всех отправляемых данных.

Чтобы все получатели смогли получить все данные, необходимо использовать BroadcastChannel.

```
1 | val broadcastChannel = BroadcastChannel<Int>(1)
```

В конструкторе нужно указать capacity. Этот тот же параметр, что мы уже разбирали для обычного канала. Но с парой отличий. Значение по умолчанию 0 здесь уже не подойдет. Т.е. размер буфера должен быть минимум 1. А также нельзя использовать безлимит - UNLIMITED.

BroadcastChannel наследует интерфейс SendChannel. Поэтому у него есть метод send для отправки данных.

```
1 | broadcastChannel.send(5)
```

А каждый, кто собирается получать данные, может создать себе отдельный ReceiveChannel методом openSubscription.

```
1 | val channel = broadcastChannel.openSubscription()
```

И с этого канала уже можно получать данные обычным способом: receive, for, consume. В каждый такой канал BroadcastChannel будет переправлять все данные, которые мы отправим ему методом send. И каждый получатель, создавший себе свой ReceiveChannel получит все эти данные.

Есть пара особенностей, о которых имеет смысл упомянуть.

Когда мы создаем себе канал методом openSubscription, мы уже не получим в этот канал данные, отправленные ранее. Даже если они все еще лежат в буфере.

При отправке данных в обычный канал, мы знаем, что отправка будет ждать, пока получатель не заберет данные или пока не освободится место в буфере. Отправка данных в BroadcastChannel работает аналогично. Она ждет, пока во всех созданных методом openSubscription каналах получатели не заберут данные. Только, когда данные ушли всем, тогда можно отправлять следующие данные.